Paiflow setup for macOS

Adapted for BAYANIHAN | October 24, 2026

This guide replaces the supplied Windows setup instructions with macOS Terminal instructions. It covers Intel and Apple Silicon Macs using zsh. The Claude Code section requires macOS 13 or newer according to its current documentation.

These are instructions for when you choose to start building. No installation, starter clone, account signup or payment has been performed as part of preparing this guide.

1. Terminal and Git

Open Terminal from Applications > Utilities, or search for Terminal with Spotlight. Use Command+V to paste. Run one block at a time and check its result. Use Control+C to stop a running development server.

Check the shell and Git:

```
echo "$SHELL"
git --version
```

The default macOS shell is `/bin/zsh`. These instructions use `~/.zshrc`, not Windows Git Bash's `~/.bashrc`.

If Git prints a version, continue. If macOS offers to install Command Line Tools, complete that installation. You can also request it explicitly:

```
xcode-select --install
```

Wait for the installer to finish, reopen Terminal, and run `git --version` again. Full Xcode is not needed for this web-app setup.

Checkpoint: Git prints a version. There is no need for Git for Windows, Git Bash, winget or Windows PATH commands.

If you use a custom bash or fish setup, open a zsh session before following the shell-specific commands below. Do not change your shell configuration merely to run the guide.

2. Node.js and pnpm

The supplied Paiflow setup requires Node `>=22.22.2 <23` and pnpm `10.4.1`. Pinning Node 22.22.2 reproduces the guide. A newer Node 22.x also fits the stated range; Node 23/24 does not.

Check existing tools:

```
node --version
npm --version
command -v nvm
pnpm --version
```

If Node already meets the range and npm works, skip nvm installation. If nvm is already available, skip its installer and select the Node version below. macOS uses `nvm-sh/nvm`, not NVM for Windows.

To install nvm in zsh, following the official nvm installer:

```
touch ~/.zshrc
curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh -o \
  /tmp/paiflow-nvm-install.sh
PROFILE="$HOME/.zshrc" bash /tmp/paiflow-nvm-install.sh
source ~/.zshrc
command -v nvm
```

Select the supported runtime:

```
nvm install 22.22.2
nvm use 22.22.2
node --version
npm --version
```

Install pnpm only if the existing version is not 10.4.1:

```
npm install --global pnpm@10.4.1
pnpm --version
```

Checkpoint: Node prints v22.22.2 (or another supported 22.x), and pnpm prints 10.4.1. With nvm, no sudo is needed for this npm install. In a new terminal, use `nvm use 22.22.2` before working on the starter. On Apple Silicon, use a native terminal rather than mixing Rosetta and native Node installations.

3. Run the starter when building begins

This section creates files. Run it only when you are ready to begin implementation. Keep the current starter's payment/recovery logic as the integration baseline.

First-time clone:

```
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/artisam-paiflow/paiflow-hackathon-starter.git
cd ~/projects/paiflow-hackathon-starter
nvm use 22.22.2
pnpm install --frozen-lockfile
```

If the folder already exists, open the existing checkout; do not clone over it. If you use another compatible Node manager, omit the nvm command and verify its selected version instead.

For the first setup only, create the local environment file:

```
cp .env.example .env.local
pnpm dev
```

Do not run the copy command over an already configured `.env.local`. Open the address printed by the server, usually `http://localhost:3000`. Payment actions remain disabled until valid deployment settings are supplied.

Checkpoint: the starter page loads. Stop it with Control+C before continuing with configuration. You can restart it later with `pnpm dev`.

The Windows guide deletes the cloned `.git` directory. That is not required to run the starter. This adaptation retains history and the original remote for now. When publishing your own project, deliberately configure your own repository destination and review `git remote -v` before pushing. Repository publishing is outside this setup exercise.

Do not copy malformed URLs or operators from extracted PDF text. The commands here use ordinary `https://` URLs and native macOS paths.

4. Wallets and the BAYANIHAN flow

Install Freighter from its official website in a supported browser. Use that same browser for Paiflow and the local app. Choose Stellar Testnet.

Create dedicated testnet accounts for Customer, Worker and BAYANIHAN Platform. Fund each with Friendbot's testnet XLM, leaving enough for fees and account reserves. A recipient account can also act as deployer. Use distinct worker and platform addresses.

Sign in at Paiflow with Google. If asked for an access code, use the organiser-supplied signup code privately. The signup code is different from the developer token; it is not an app setting.

In the Paiflow builder:

- Add On Receive and connect it to Split.
- Select native XLM throughout the flow.
- Set the worker's public address to 85% and the platform's to 15%.
- Review the addresses, asset, network, percentages and deployment fee.
- Deploy with the funded deployer wallet and wait for CONFIRMED.
- Copy the deployment UUID, not its flow ID or C-prefixed contract address.

Open the confirmed deployment's hosted payment page. Connect the Customer account, review a 1-XLM testnet payment and sign in Freighter. Expected payout: 0.85 XLM to Worker and 0.15 XLM to Platform, with network fees additional to the customer's payment.

Save the transaction hash and verify both recipients and amounts in the deployment activity or Stellar Expert Testnet. A slow activity feed is not a reason to pay twice.

This adapts the supplied tutorial's 90/10 shop/community example to the deck's 15% commission. It is an immediate split when payment executes, not escrow. BAYANIHAN's proposed app asks the customer to pay after confirming completion.

Checkpoint: a confirmed testnet deployment and a verified hosted-page payment exist before app integration starts. Keep wallet recovery phrases and secret keys in Freighter; the app only needs public wallet addresses.

5. Configure and verify the SDK payment

In Paiflow Account > Developer, create a Read and operate token. Explicitly grant it access to the confirmed deployment. Copy its one-time secret into the server environment file. A developer token starts with `pfd_`; the supported deployment-bound alternative starts with `pfk_`.

Set these values in the starter's `.env.local` using your editor:

```
PAIFLOW_BASE_URL=https://beta.app.paiflow.xyz
PAIFLOW_DEPLOYMENT_ID=YOUR_CONFIRMED_DEPLOYMENT_UUID
PAIFLOW_API_TOKEN=YOUR_DEVELOPER_TOKEN
```

These values are server configuration. Do not prefix them with `NEXT_PUBLIC_`. Check that `.env.local` is ignored before committing:

```
git check-ignore .env.local
pnpm dev
```

The ignore check should print `.env.local`. If it does not, add that filename to `.gitignore` before committing anything. Restart the server whenever you change environment values.

Connect the Customer wallet on Testnet. Review the 1-XLM payment, sign in Freighter, and check the original transaction. Verify the exact 0.85/0.15 payouts separately. Keep its hash for the demo.

The starter already pins `@paiflow/sdk@0.2.0`. For a separate app, the guide's install command is `pnpm add @paiflow/sdk@0.2.0`. Do not replace the explicit version with an unversioned install; the supplied guide warns that `latest` may resolve to the older SDK. SDK code runs on the Node server; Freighter signing runs in the browser.

Recovery: after a timeout, UNKNOWN, NOT_FOUND, expired preparation or delayed event, check the saved original transaction. Do not clear browser storage, switch deployment or create another payment just to remove the pending state. Token removal does not cancel an already submitted transaction.

Application work still required: the starter does not include durable booking records, customer authentication, fulfilment or independent split verification. Add those before the booking app marks an order paid. HTTP 200 or a generic SUCCESS label alone does not verify the expected recipients.

6. Claude Code and Engr on macOS

This preserves the Windows guide's optional coding-assistant workflow. Engr and Kimi are development tools, not runtime dependencies of the booking app or Paiflow payment integration.

Check first:

```
claude --version
engr --help
```

If Claude Code is missing, use its official macOS installer:

```
curl -fsSL https://claude.ai/install.sh -o /tmp/paiflow-claude-install.sh
bash /tmp/paiflow-claude-install.sh
```

Open a new Terminal window and check `claude --version`. If the command is missing, add the native installation directory to zsh's PATH as shown below. The native installer requires macOS 13 or newer under current documentation.

Install Engr outside the app folder. If `~/engr` is already your Engr checkout, skip the clone:

```
git clone https://github.com/kimerran/engr.git ~/engr
bash ~/engr/install
```

If `~/.local/bin` is not already configured in `~/.zshrc`, add it once:

```
printf '\nexport PATH="$HOME/.local/bin:$PATH"\n' >> ~/.zshrc
source ~/.zshrc
claude --version
engr --help
```

Checkpoint: both commands respond. No winget or Windows-specific installer is involved. The Engr repository is separate from the BAYANIHAN app checkout.

If the installer or provider rejects an environment, use the exact diagnostic to resolve it; successful installation does not prove a Kimi key has valid provider access.

7. Use Kimi from zsh

Check whether a key is present without displaying it:

```
if [[ -n "${ENGR_KIMI_API_KEY:-}" ]]; then
  export ENGR_KIMI_API_KEY
  printf 'Kimi key is configured\n'
else
  printf 'Kimi key is not set\n'
fi
```

If it is not set, use this zsh-specific hidden prompt. Paste the full block, then enter only the key when prompted:

```
read -rs 'ENGR_KIMI_API_KEY?Kimi API key: '
printf '\n'
if [[ -n "$ENGR_KIMI_API_KEY" ]]; then
  export ENGR_KIMI_API_KEY
  printf 'Key set for this terminal session\n'
else
  unset ENGR_KIMI_API_KEY
  printf 'No key entered\n'
fi
```

This avoids putting the key into command history or app files. Unlike the Windows guide, it does not append a plaintext key to a shell startup file. Enter the key again in each new coding terminal, or use an existing trusted secret-management setup.

When the app checkout exists, open it before starting Engr:

```
cd ~/projects/paiflow-hackathon-starter
```

Only if that directory change succeeds, run:

```
engr --kimi
```

In another Terminal window, open the same folder, select Node 22.22.2 and run `pnpm dev` to preview the app. Review the assistant's requested actions. The Kimi key and Paiflow token serve different purposes; neither belongs in screenshots, source control or chat prompts.

Suggested first instruction when implementation is authorised: “Read README.md and AGENTS.md. Explain the payment and recovery code before adapting the booking flow.” During planning, do not ask the assistant to start building.

8. Troubleshooting and readiness

nvm not found: run `source ~/.zshrc`, then `command -v nvm`. The result should be `nvm`; it is a shell function. If needed, check the official nvm installer instructions for the profile it modified.

Wrong Node or missing pnpm: run `nvm use 22.22.2`, then `which node`, `node --version` and `pnpm --version`. Install pnpm 10.4.1 under that selected Node. Avoid sudo or Windows PATH recipes.

Apple Silicon architecture mismatch: check `uname -m` and `node -p process.arch`. Native Apple Silicon normally reports `arm64` for both. Reopen a native terminal and reinstall/select Node through nvm if you mixed architectures.

Dependency installation fails: check network access and runtime versions. Keep the lockfile; ask for help with the exact error instead of deleting it or upgrading everything.

claude or engr not found: reopen Terminal, source `~/.zshrc`, and check that `~/.local/bin` is on PATH. If an old Engr checkout lacks `--kimi`, inspect its help/version and repository instructions before updating it.

Kimi key rejected: verify the provider key is active and appropriate for the configured endpoint. Re-enter it with the hidden prompt. Shell presence does not establish provider validity.

Port 3000 busy: use the address printed by the dev server or stop your previous server with Control+C. **Payment disabled:** check all three server environment settings and restart. **401/403/404:** check token expiry, permissions, explicit deployment grant and the UUID. **429:** respect Retry-After.

Ready: Git works; Node meets the supported 22.x range; pnpm is 10.4.1; the starter loads; wallets are funded on Testnet; the flow is confirmed; a hosted and an SDK payment have verified expected payouts; original hashes are retained. Claude/Engr readiness is optional for running the app.

References

- Supplied setup-guide.pdf, paiflow-guide.pdf and event-primer.pdf; adapted October 24, 2026.
- Paiflow starter
- Paiflow SDK
- nvm for macOS/Linux
- Apple Command Line Tools
- Claude Code setup
- Engr
- Freighter
- Paiflow app
- Stellar Expert Testnet

Commands were reviewed and syntax-checked where applicable; they were not executed to install software or create an app. Live sign-in, provider credentials and payment execution remain future setup checks.